# Clase 10 — DNS: la base de datos distribuida

**Bloque:** B (Aplicación) · **Semana:** 4 · **Duración estimada:** 110 min **Correlativa:** Clase 9 (HTTP a fondo) · **Sigue con:** Clase 11 (DNS: iterativo vs. recursivo, caché y TTL) **Material de referencia:** Tanenbaum cap. 7 (§7.4) · RFC 1034, RFC 1035, RFC 4033-4035, RFC 5155, RFC 6891, RFC 8659, RFC 9460

---

## 🎯 Al terminar esta clase vas a poder

1. Explicar **por qué existe** el DNS y por qué no puede resolverse con un archivo central ni con una tabla plana en cada host.
2. Explicar por qué el DNS es **dos cosas a la vez** (espacio de nombres jerárquico y base de datos distribuida) y por qué el espacio se resuelve **de derecha a izquierda**.
3. Dibujar la **jerarquía** y decir **qué sabe cada tipo de servidor**: el root, el TLD, el autoritativo y el local.
4. Explicar cómo funciona la **delegación** con registros `NS` y `glue`, y por qué existe la **paradoja del bootstrap**.
5. Escribir un **Resource Record** completo con sus **cinco campos**, y decir qué tamaño tiene cada uno.
6. Distinguir los **tipos de RR** que hay que saber ( `A` , `AAAA` , `NS` , `CNAME` , `MX` , `SOA` , `PTR` , `TXT` , `SRV` , `CAA` ) diciendo qué resuelve cada uno.
7. Resolver completo el **Examen 2010-02 Pregunta 1a** (qué es DNS y su función, 8 puntos).

> **Prerequisito del examen:** `2010-02 P1` (8 puntos — ★ qué es el DNS y su función), `2010-12 P1a` (8 puntos — el RR y sus cinco campos, que es el otro punto de esta clase), `2007-07 P4` (función del DNS y protocolo involucrado).

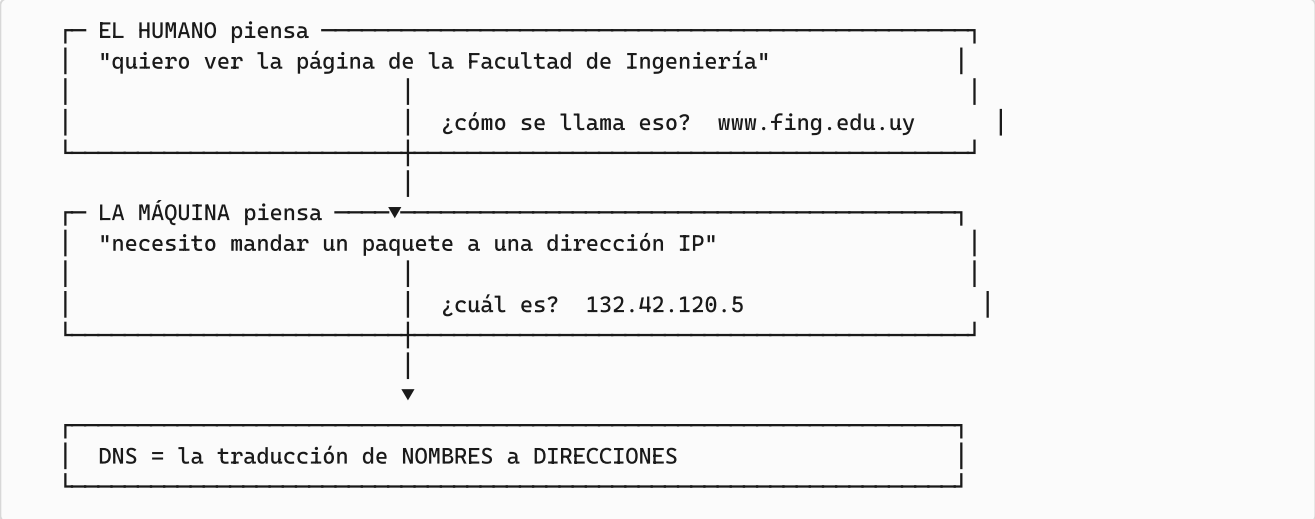
---

# PARTE 1 — TEORÍA

## 1.1 El problema que resuelve el DNS

Los humanos no memoramos direcciones IP. No es que no podamos: es que **una dirección IP no significa nada para una persona**. `132.42.120.5` no te dice quién es, no te dice qué organización lo administra, no te dice si el servidor está encendido, y no te sirve de nada si mañana lo mueven de datacenter. En cambio `www.fing.edu.uy` te dice todo eso de un vistazo.

El DNS es la capa que hace esa traducción:




**La objeción obvia**: ¿no podría ser un archivo de texto con todas las correspondencias, copiado a cada máquina? No, y hay **tres razones independientes** que lo impiden. Cada una basta sola.

### Razón 1 — La escala: la tabla no entra

Un archivo central tendría que contener **una entrada por cada host de Internet**:

```
En 2011 (el curso):   ~2.000 millones de hosts
En 2026:              ~5.000 millones de hosts      (verificar en vivo)

Cada entrada necesita, mínimo:
  nombre (promedio 15 bytes) + IP (4 bytes) + tipo (2) + TTL (4)
  ≈ 25 bytes mínimo, y en la práctica 60–80 con las líneas completas

5.000 millones × 60 bytes = 300.000.000 bytes = 300 MB POR CADA MÁQUINA
```

Y encima hay que **actualizarlo**: si un host aparece, el archivo tiene que volver a distribuirse. Un archivo de 300 MB que se actualiza constantemente es imposible de mantener.

### Razón 2 — El punto único de falla

```
┌──────────────┐
│  ARCHIVO     │◄── todos los hosts del mundo lo consultan
│  CENTRAL     │
└──────────────┘
      │
  si se cae  ──►  TODO Internet deja de resolver nombres

  si alguien lo compromete ──►  puede REDIRIGIR cualquier nombre
                                a cualquier IP  (phishing perfecto)
```

Un archivo central es un **punto único de falla** (si cae, cae todo) y un **punto único de compromiso** (si lo hackean, controlás la resolución de todo Internet). Las dos cosas son inaceptables.

### Razón 3 — El mantenimiento

Hay que responder **quién** autorizó a registrar cada nombre. Un archivo central no tiene forma de saber si `fing.edu.uy` le pertenece a la Universidad o a un impostor.

**La solución del DNS ataca las tres:**

| Problema | Solución del DNS |
|---|---|
| **Escala** (300 MB por host) | **distribución**: cada servidor guarda solo la porción que le toca, y los nombres están **particionados** por dominio |
| **Punto único de falla** | **replicación y jerarquía**: los servidores root están replicados; cada zona tiene varios servidores autoritativos |
| **Mantenimiento** | **delegación**: cada zona delega la zona siguiente a una autoridad identificada, y esa autoridad es la que valida |

## 1.2 El DNS son dos cosas a la vez

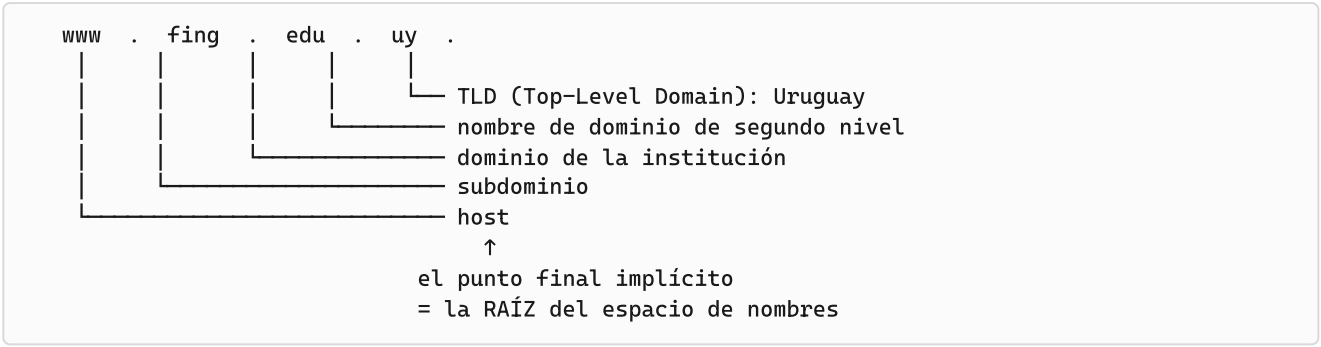
Esta es la idea central de la clase y lo que el examen 2010-02 P1 pide exactamente:

```
╔═══════════════════════════════════════════════════════════╗
║  EL DNS ES DOS COSAS A LA VEZ:                            ║
║                                                           ║
║  (a) UN ESPACIO DE NOMBRES JERÁRQUICO                     ║
║      → la estructura de los NOMBRES: cómo se nombran las cosas ║
║      y cómo se reparte la autoridad sobre ellos           ║
║                                                           ║
║  (b) UNA BASE DE DATOS DISTRIBUIDA                        ║
║      → el contenido: qué nombre corresponde a qué dirección ║
║      y cómo se consulta esa correspondencia               ║
╚═══════════════════════════════════════════════════════════╝
```

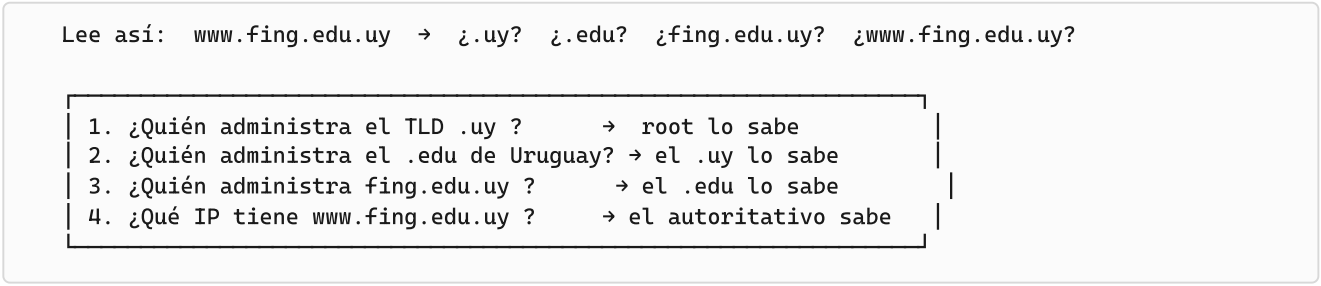
**Y son dos cosas distintas con dos respuestas distintas.** La primera es la que te dice **a quién preguntarle**; la segunda es la que te dice **qué preguntar**. El `2010-02 P1` lo dice literal en su solución oficial: "*protocolo de capa de aplicación que permite realizar la traducción de nombres de hosts a direcciones IP. También se denomina DNS a la base de datos jerárquica distribuida que almacena los mapeos entre nombres y direcciones IP*".

### (a) El espacio de nombres: jerárquico y de derecha a izquierda

El nombre completo de un host es una **etiqueta jerárquica separada por puntos**, con un punto implícito al final:



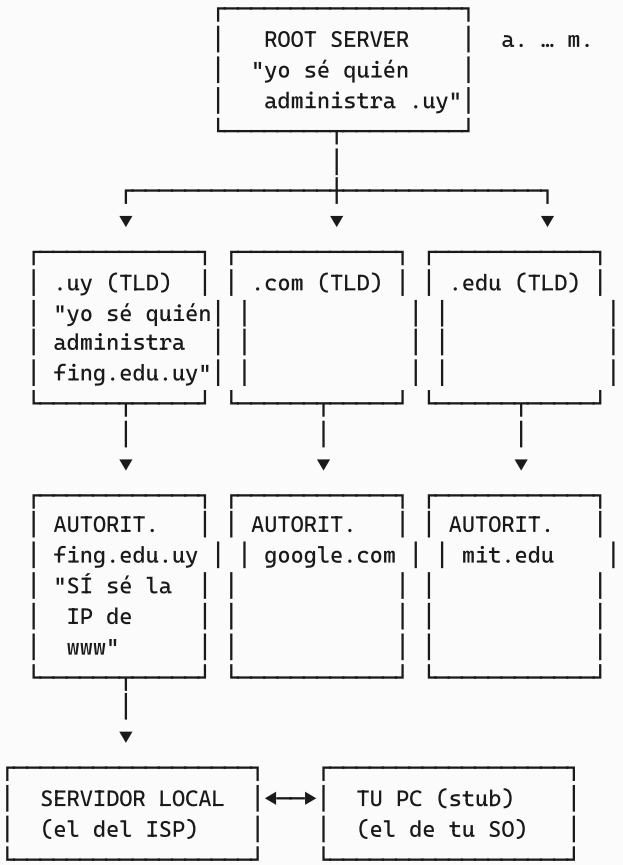

**Se resuelve de derecha a izquierda**, y esto es contraintuitivo para un lector de español, donde leemos de izquierda a derecha. La razón es histórica y es la estructura de la autoridad:




**Por qué de derecha a izquierda y no al revés:** la **autoridad va de arriba hacia abajo**. El root sabe qué TLDs existen; el TLD sabe qué dominios de segundo nivel hay debajo; el autoritativo del dominio sabe qué hosts hay. Cada nivel **solo conoce un nivel hacia abajo**, y por eso el árbol tiene que recorrerse desde la raíz, que es el único que sabe dónde empieza todo. Si el nombre se recorriera de izquierda a derecha, el primer nivel («qué hay bajo `www`?») no lo sabe nadie: `www` no es una zona.

**Y `uy` a la derecha de `edu` significa que `edu` es de Uruguay, no el `edu` de Estados Unidos.** Los TLD countries están estructurados así: `fing.edu.uy` (Uruguay) y `mit.edu` (Estados Unidos) son dos ramas distintas de `.edu`. Esta estructura es la que hace posible la **delegación por país sin cambiar la infraestructura global**.

## 1.3 Los tipos de servidores, con su rol exacto




### Root server: 13 identidades lógicas, replicadas por anycast

| Dato | Detalle |
|---|---|
| **Cuántos hay** | **13 identidades lógicas**, nombradas de `a` a `m`: `a.root-servers.net` … `m.root-servers.net` |
| **Cuántas máquinas** | **cientos**: cada identidad se opera en **cualquiercast**, así que hay una IP pública igual en todo el mundo |
| **Cuántas IPv4 por identidad** | **cuatro** direcciones IPv4 por identidad, por *anycast* |
| **Operación** | los root servers los financian los operadores de la red (los *NIC*), con cable submarino; no hay un actor comercial detrás |
| **Qué SÍ saben** | la lista de los **TLD** y cuál es la dirección IP de cada servidor TLD |
| **Qué NO saben** | **no conocen los dominios**. Jamás supieron qué es `fing.edu.uy`. |

> **El punto que más se confunde y más se pregunta:** el root **no resuelve nombres de dominio**. Solo tiene una tabla plana y pequeña de unos 1500 TLDs. Si le preguntás por `www.fing.edu.uy`, te contesta «no sé, pero el `.uy` está en `X.Y.W.Z`».

**Por qué 13.** El número original (1987) fue un capricho del DARPA, y hoy está congelado porque el *root zone* está firmado digitalmente: hay que coordinarse con los operadores de los 13 para cambiar **cualquier** registro de la zona raíz. **El número es un artefacto histórico, no una decisión de diseño**, y es un buen ejemplo de *protocolos que no se pueden cambiar*.

### TLD server: el `.uy`, el `.com`, el `.edu`

| Dato | Detalle |
|---|---|
| **Qué es** | un dominio ( `uy` , `com` , `edu` , `org` , `ar` , `io` ...) |
| **Qué SÍ sabe** | los **registros** `NS` de los dominios de segundo nivel bajo él. Del TLD `.uy` sabe quién administra `fing.edu.uy` . |
| **Qué NO sabe** | **tampoco conoce los dominios**, ni mucho menos sus IPs. Jamás supo qué es `www.fing.edu.uy` . |
| **Quién lo opera** | los *registries*: NIC Uruguay para `.uy` , Verisign para `.com` , ICANN para `xxx` . |
| **Cuántos** | cientos por TLD, y casi todos los **gTLD** son «genéricos» ( `.com` , `.org` , `.info` ...), o sea los que no son de un país |

### Servidor autoritativo: el que sí sabe

Es el **dueño de la información**. Preguntale a `ns1.fing.edu.uy` y te contesta la IP de `www.fing.edu.uy` con total seguridad.

| Dato | Detalle |
|---|---|
| **Qué es** | el servidor del dominio: `ns1.fing.edu.uy` , `ns2.fing.edu.uy` |
| **Qué sabe** | **TODOS los RRs de su zona**: A, AAAA, MX, TXT, CNAME, SRV... |
| **Es la fuente de verdad** | sí. Si el autoritativo dice `X` , `X` es. Todos los demás son copias. |
| **Cuántos hay** | mínimo 2 por zona, si posible en **redes distintas** y en **ciudades distintas** |
| **Quién lo pone** | el administrador del dominio |

### Servidor de nombres local: recursivo, y el stub resolver del host

Acá está el punto donde la cátedra más hace líos, así que hay que separar **dos cosas muy distintas** que a veces se confunden:

| | **Stub resolver** (el de tu PC) | **Servidor de nombres local / recursivo** (el del ISP) |
|---|---|---|
| **Qué es** | una biblioteca diminuta dentro del **sistema operativo** | un programa `named` o `dnsmasq` en el servidor del ISP o del router |
| **Dónde está** | en **tu PC** | en la **red**: la red de tu ISP, o el router de tu casa |
| **Qué sabe** | **nada**. No mantiene caché propia (aunque Windows sí guarda un archivo local, `dnsapi.dll`). | **mantiene caché** de todo lo que resolvió, con TTL |
| **Qué hace** | agarra el nombre y se lo **manda al recursivo** | **pregunta a root, TLD y autoritativo** y **cachea** |
| **Consume CPU del host** | casi nada | todo el trabajo |

> **Mnemotecnia:** *stub* significa «trozo», «pieza mínima». *Resolver* es «resolver», el que resuelve. El stub **no resuelve**: **pregunta**. El recursivo **resuelve**: **pregunta por vos**.

## 1.4 Delegación: `NS` y `glue records`

La delegación es el mecanismo por el que **el root delega `.uy`, y `.uy` delega `fing.edu.uy`**. No es un aviso verbal: es un **registro** en la base de datos del nivel superior.

### El mecanismo: un registro `NS` en el nivel superior

```
En la zona del TLD .uy hay:

$ORIGIN uy.
uy.         3600  IN  NS  a.nic.uy.
uy.         3600  IN  NS  b.nic.uy.

En la zona de fing.edu.uy hay:

$ORIGIN fing.edu.uy.
fing.edu.uy.     3600  IN  NS  ns1.fing.edu.uy.
fing.edu.uy.     3600  IN  NS  ns2.fing.edu.uy.
www        A     3600  IN  A   132.42.120.5
```

Cuando le preguntás al root por `www.fing.edu.uy`, el root no tiene el `A`, pero **sí tiene el `NS` de `uy`**. Te lo pasa como **referencia** («buscale al `.uy`») y con eso te ahorró el trabajo de preguntar.

### La paradoja del bootstrap: por qué existen los *glue records*

Y acá aparece el problema elegante que hay que explicar bien:

```
El .uy te dice:  "el autoritativo de fing.edu.uy es ns1.fing.edu.uy"
↑
¡PERO para preguntarle a ns1.fing.edu.uy
  NECESITO SU DIRECCIÓN IP!
Y esa IP es... ¡otro nombre que hay que resolver!
```

**Esto es un círculo**: para resolver un nombre (el del servidor autoritativo) hay que resolver un nombre (el que te dijeron). La solución se llama **bootstrap**, y el truco es guardarse el resultado en el mismo lugar de donde lo obtuviste.

```
En la zona del TLD .uy, ADEMÁS del NS, están los GLUE RECORDS:

$ORIGIN uy.
uy.        3600  IN  NS  a.nic.uy.
...
ns1.fing.edu.uy.  3600  IN  A   132.42.60.1    ⟵ GLUE
ns2.fing.edu.uy.  3600  IN  A   132.42.60.2    ⟵ GLUE
```

**El `glue record` es un registro `A` (o `AAAA`) con el nombre del servidor de nombres, guardado en el servidor del nivel superior.** Y es **redundante**, y esa redundancia es intencional: el mismo registro `A` para `ns1.fing.edu.uy` existe **también** en la zona `fing.edu.uy`, que es donde le corresponde. El problema es que **para llegar a esa zona hay que pasar por el glue**, así que el glue tiene que estar en el nivel superior.

> **La regla que hay que memorizar:** el *glue* es **obligatorio** solo para los nombres de servidor que están **dentro de la zona delegada**. Para un servidor de nombres **fuera** del dominio ( `ns1.otroproveedor.com` ), **no hay glue**, porque su nombre se resuelve por el camino normal, con otro nivel de por medio.

**Los problemas operativos del glue, que son los que aparecen en la práctica:**

1. **Desincronización.** El `glue` dice `132.42.60.1`, pero el autoritativo cambió a `132.42.60.9`. El glue está **desactualizado** y toda resolución de `fing.edu.uy` falla hasta que el TTL del glue venza. Es la causa número uno de «mi dominio dejó de funcionar después de un cambio».
2. **Servidores que se caen.** Si `ns1.fing.edu.uy` baja, todo el dominio baja: los clientes prueban `ns2`, pero la resolución del *nombre* `ns2.fing.edu.uy` necesita el glue, que tiene el mismo TTL.
3. **La respuesta mínima.** Algunos resolvers de producción devuelven solo los registros **de la zona que se consultó** y **no** los *glue*, obligando a un segundo viaje.

## 1.5 El Resource Record: la unidad de información

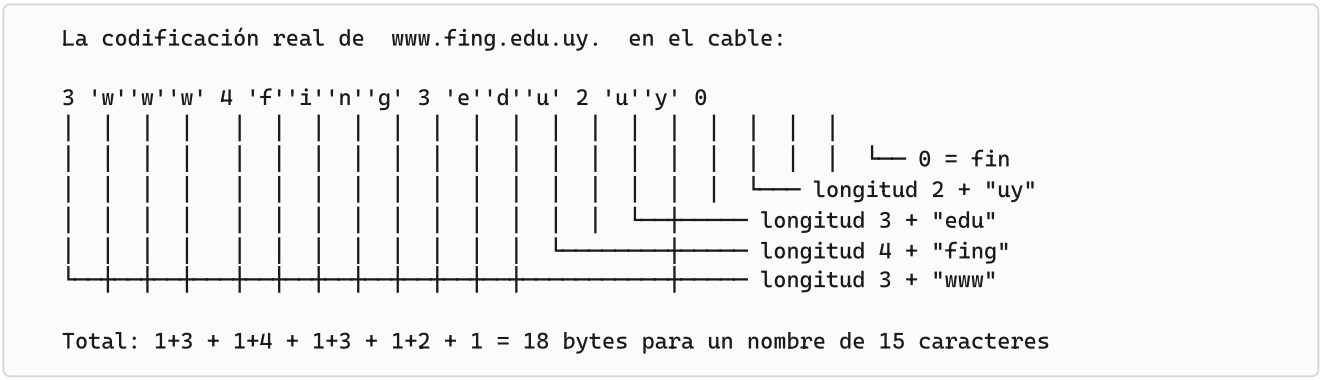
Todo lo que el DNS sabe son **Resource Records** (*RR*, registros de recurso). Un RR es una **línea** de la base de datos, y tiene **cinco campos**:

```
nombre              clase   tipo    TTL      datos
fing.edu.uy.        IN      A       3600     132.42.120.5
----------------    -----   ----    -----    ------------------
nombre             2 bytes  2 bytes 4 bytes  variable
```

| # | Campo | Tamaño | Qué es | Ejemplo |
|---|---|---|---|---|
| 1 | **Nombre** (*name*) | 1 byte por etiqueta + 1 byte de longitud | el nombre del recurso, **en forma absoluta**: siempre termina en punto | `fing.edu.uy.` |
| 2 | **Clase** (*class*) | 2 bytes | casi siempre `IN` (*Internet*). Existen `CH` (CHAOS, para el `version.bind`) y `HS` (obsoleto) | `IN` |
| 3 | **Tipo** (*type*) | 2 bytes | qué clase de dato es | `A` |
| 4 | **TTL** (*time to live*) | 4 bytes | segundos que el registro se puede cachear | `3600` |
| 5 | **Datos** (*rdata*) | **variable** | el valor, y **su formato depende del tipo** | `132.42.120.5` |

**Tres cosas que hay que entender de los nombres, porque son las trampas del examen:**

1. **El punto final es obligatorio** en la forma canónica. `fing.edu.uy` y `fing.edu.uy.` son **el mismo nombre**: el último punto es implícito.
2. **Las etiquetas se cuentan de derecha a izquierda.** El nombre `fing.edu.uy` tiene **3 etiquetas**; `www.fing.edu.uy` tiene **4**.
3. **Un nombre de longitud total mayor a 255 bytes no existe.** El nombre se codifica como una secuencia de etiquetas de hasta 63 bytes cada una, precedidas por un byte de longitud: `7fing3edu2uy0` son 1+5+1+3+1+2+1 = 14 bytes. Esa codificación es la razón por la que **los nombres legibles son solo una convención de presentación**: en el cable viajan como longitud + caracteres.




**Y el `2010-12 P1a` es exactamente esto**, que es el otro problema central de esta clase:

> **Examen 2010-12, Pregunta 1a:** *«Cuando un servidor de nombres aprende un mapeo, lo cachea. ¿Cuál es el campo del registro de recurso (RR) que decide si se puede cachear o no? ¿Cuál es el servidor que define ese tiempo? ¿El valor que toma ese campo, a qué refiere y en qué unidades está expresado?»*

Se resuelve **entero** en la Clase 11 (que desarrolla caché y TTL), pero el **campo** ya está definido acá: es el **cuarto campo**, el **TTL**, de **4 bytes**, y su unidad son **segundos**. Lo que se responde completo en la Parte 3 de esta clase.

## 1.6 Los tipos de Resource Record que hay que saber

| Tipo | Qué resuelve | Ejemplo | Notas y trampas |
|---|---|---|---|
| `A` | nombre → **IPv4** | `www.fing.edu.uy. IN A 132.42.120.5` | 4 bytes de datos. Varios `A` = **balanceo por round-robin** (el orden lo elige el cliente; Chrome y Safari lo aleatorizan). |
| `AAAA` | nombre → **IPv6** | `www.fing.edu.uy. IN AAAA 2001:7fd::1` | 16 bytes. El *happy eyeballs* es un cliente que consulta `A` y `AAAA` **en paralelo** y se queda con la que llegue primero. |
| `NS` | nombre → **servidor de nombres** autoritativo | `fing.edu.uy. IN NS ns1.fing.edu.uy.` | El `NS` **no contiene la IP**: solo el nombre del servidor. La IP viene en el glue. **Es lo que hace funcionar la delegación.** |
| `CNAME` | **alias**: nombre → **otro nombre** | `www IN CNAME web-server01.uy.` | Solo puede haber **uno por nombre**. No puede existir en la raíz. **No puede coexistir con otros tipos en el mismo nombre.** |
| `MX` | nombre → **servidor de correo**, con **preferencia** | `fing.edu.uy. IN MX 10 mail.fing.edu.uy.` | El número (aquí `10`) es la **preferencia**: menor = más preferido. El que tiene menor MX es al que se le manda primero. Si cae, se prueba el siguiente. |
| `SOA` | **Start of Authority**: datos de **autoridad de la zona** | `fing.edu.uy. IN SOA ns1.fing.edu.uy. admin.fing.edu.uy. 2026100601 7200 3600 1209600 3600` | Lleva **MNAME**, **RNAME**, **serial**, **refresh**, **retry**, **expire** y **minimum**. Es el registro de **control**: los servidores secundarios lo usan para decidir cuándo resincronizar. |
| `PTR` | **IP → nombre**, la inversa | `5.120.42.132.in-addr.arpa. IN PTR www.fing.edu.uy.` | Se llama resolución **inversa** y vive en `.in-addr.arpa` (IPv4) o `.ip6.arpa` (IPv6). **El nombre es al revés.** |
| `TXT` | texto libre arbitrario | `fing.edu.uy. IN TXT "v=spf1 include:_spf.fing.edu.uy -all"` | Se usa para **SPF** (qué servidores pueden enviar correo en tu nombre) y para **verificación de propiedad** de dominios ante proveedores de correo. |
| `SRV` | nombre → **servicio con sus parámetros de puerto y prioridad** | `_sip._tcp.fing.edu.uy. IN SRV 10 20 5060 sip.fing.edu.uy.` | Formato: `prioridad peso puerto objetivo`. Se inventó para **SIP** y hoy se usa en XMPP, Matrix, FTP. No existe en RFC 1034: es una extensión. |
| `CAA` | **qué Autoridades de Certificación** pueden emitir para el dominio | `fing.edu.uy. IN CAA 0 issue "letsencrypt.org"` | `RFC 8659` (2019), reemplaza a `RFC 6844`. Evita que una CA emita un certificado para tu dominio sin tu permiso. |

## Los tres que se confunden siempre

**`CNAME` vs. `MX` vs. `SOA`** — son tres respuestas a tres preguntas distintas:

```
¿A qué IP va un cliente que abre http://www.fing.edu.uy ?
   ──► CNAME (si existe) dice:  "ese nombre es otro nombre"
   ──► A       dice:          "esa IP es esta"
   ──► SOA    NO se usa para esto.

¿A qué servidor mando el correo a admin@fing.edu.uy ?
   ──► MX  dice:  "preferencia 10, servidor mail.fing.edu.uy"
   ──► y después, para resolver mail.fing.edu.uy → IP, se usa CNAME o A.

¿Quién manda aquí y cada cuánto sincronizo ?
   ──► SOA dice:  "el master es ns1, el serial es 2026100601"
```

**El `SOA` en detalle, porque tiene que saberse de memoria:**

```
   fing.edu.uy.  IN  SOA  ns1.fing.edu.uy.  admin.fing.edu.uy.  (
                                 |            |            |
                                 |            |            └─ 1: SERIAL
                                 |            |               (el "número de versión"
                                 |            |                de la zona)
                                 |            └─ 2: RNAME
                                 |               (el email del administrador,
                                 |                con el @ como punto)
                                 └─ 1: MNAME
                                    (el servidor de nombres master)

                2026100601    7200    3600    1209600    3600
                     |          |       |         |         |
                     |          |       |         |         └── 7: MINIMUM
                     |          |       |         |             (TTL por defecto de los
                     |          |       |         |              NXDOMAIN negativos)
                     |          |       |         └── 6: EXPIRE
                     |          |       |            (días antes de dejar de
                     |          |       |             intentar más refreshing)
                     |          |       └── 5: RETRY
                     |          |          (minutos entre reintentos de refresh
                     |          |           cuando falla)
                     |          └── 4: REFRESH
                     |             (minutos entre que un secundario
                     |              pregunta "¿hay cambios?")
                     └── SERIAL
   ```

**Y la comparación `CNAME` vs. `MX`, que es pregunta clásica:**

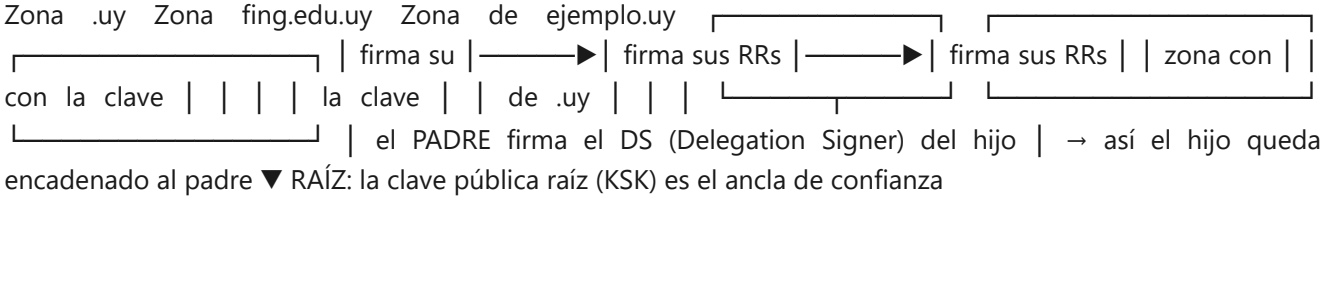
| | `CNAME` | `MX` |
|---|---|---|
| **Apunta a** | otro **nombre** | un **servidor de correo**, con número de preferencia |
| **Se usa para** | mover un host sin romper los enlaces | elegir a qué servidor va el correo |
| **¿Puede haber dos?** | **no**: un CNAME por nombre, y no puede convivir con nada más | **sí**: |
| **Ejemplo real** | ` intranet.fing.edu.uy → CNAME SharePoint.uy.` | ` fing.edu.uy MX 10 smtp1.fi |

## 1.7 ⚠️ Actualización a 2026 — qué cambió en el DNS desde 2011

Esta clase describe el DNS **de 2011**. Lo de arriba —jerarquía, delegación, *glue*, los cinco campos del RR, la lista de tipos— **sigue exactamente igual**. Lo que cambió es lo que se le **agregó encima**, y son cuatro cosas.

### (a) DNSSEC: la firma de la jerarquía (`RFC 4033-4035`, 2005)

**El problema que resuelve.** El DNS es, por diseño, **una base de datos abierta y confiable**: cualquiera puede registrar un dominio, y cualquiera que diga tener la delegación de un nombre es creído. Eso permite el **DNS hijacking** (secuestro de dominio) con solo lograr que el registrador o un servidor mal configurado apunten el nombre a una IP ajena. **DNSSEC** agrega criptografía: cada zona firma sus RRs con una clave privada, y la firma se valida contra una cadena que arranca en la **raíz**.



```
**El estado de despliegue, y acá está la lección de la clase:**

| Dato | Valor |
|---|---|
| Estandarizado | **2005** (`RFC 4033-4035`) |
| **gTLD firmados** | **~99,6 %** |
| **Dominios (2LD) con `DS`** | **~5–7 %** |
| Raíz firmada | desde **2010** (`RFC 5155`) |

> ⚠️ **La lección, y hay que decirla con esas palabras:** DNSSEC es el ejemplo
> perfecto de **estandarizado en 2005 y con despliegue bajo casi veinte años
> después**. La cadena criptográfica está construida y operativa, pero el último
> eslabón —el dominio de segundo nivel— casi no lo adoptó. **Por eso en 2026 no se
> puede enseñar que «el DNS es seguro»**: se puede enseñar que **el DNS tiene una
> capa de autenticidad opcional, bien diseñada y masivamente subutilizada**. El
> paralelo más cercano es HTTPS, que también lleva veinte años y tardó otras dos
> décadas en desplegarse del todo.

### (b) EDNS(0): el mecanismo que hace possible todo lo anterior (`RFC 6891`, 2013)

**El problema que resuelve.** El mensaje DNS clásico tiene **12 bytes** de cabecera
y cuatro contadores. Para extenderlo (DNSSEC, `CAA`, `SVCB`/`HTTPS`) no había
dónde: cualquier campo nuevo tenía que romper la estructura. **`EDNS(0)`** agrega un
**pseudo-registro `OPT`** en la sección *Additional* que lleva opciones de tamaño y
de extensión, **sin romper el formato antiguo**.
```

Mensaje DNS clásico Mensaje con EDNS(0)

| ID | flags | QD AN NS AR | | ID | flags | QD AN NS AR |

Question / Answer / Authority Additional:

| OPT ◀── pseudo-registro | type=41 | udp: 4096 |

```
**Importa en el examen** porque **cambia el tamaño por defecto del datagrama**:
sin `EDNS(0)` el máximo sigue siendo **512 bytes**, y las respuestas grandes se
cortan y obligan a reintentar por TCP. Con `OPT` que anuncia `udp: 4096`, el
cliente y el servidor acordan hasta 4096 bytes **sin cambiar de transporte**. Es un
ejemplo de manual de **extensión compatible hacia atrás** (*backward compatible
extension*), la misma técnica de los campos opcionales de IPv4.

### (c) CAA: el dominio declara qué autoridad puede emitir certificados (`RFC 8659`, 2019)

Un `CAA` es un RR que dice **qué entidades certificadoras** pueden emitir un
certificado para ese dominio:
```

fing.edu.uy. IN CAA 0 issue "letsencrypt.org" fing.edu.uy. IN CAA 0 issuewild "letsencrypt.org" fing.edu.uy. IN CAA 0 iodef "mailto:seguridad@fing.edu.uy"

| | | | | | | | └── a quién avisar si alguien viola la política | | | └──── qué CA puede emitir para lo que sea | | └──── "para este dominio y todos sus subdominios" | └──── 0 = "el primero de la lista manda" └──── la autoridad

```
⚠️ El curso de 2012 enseñaba **`RFC 6844`**, que quedó **obsoleta**: la vigente es
la `RFC 8659`. **Por qué importa:** sin `CAA`, cualquier autoridad de certificación
puede emitir un certificado válido para tu dominio y el navegador no tiene forma de
rechazarlo. Es uno de los **mecanismos de seguridad que se apoyan en un ancla
declarada en el DNS**, como `DNSSEC` y como `RPKI` para BGP.

### (d) El DNS pasó a transportar **política de transporte**: SVCB/HTTPS (`RFC 9460`, 2023)

Este es el cambio conceptual más grande **después** de DNSSEC, y conecta con la
Clase 9: el DNS **ya no transporta solamente direcciones**, sino **instrucciones
sobre cómo conectarse**.
```

ANTES: www.fing.edu.uy. A 132.42.120.5 "¿dónde está?" ↓ el cliente decide TODO: TCP o TLS, qué versión de HTTP

AHORA: www.fing.edu.uy. HTTPS SVCB ├─ alpn="h3,h2" ← este servidor también habla HTTP/3 ├─ ipv4hint=["132.42.120.5"] ├─ port=443 └─ ech="..." el cliente sabe QUÉ VERSIONES USAR ANTES de abrir el socket

```
**Por qué importa de verdad:** es lo que permite **descubrir HTTP/3 sin haber ido
una vez por HTTP/1.1**. Con `Alt-Svc` (Clase 9) el navegador tiene que hacer
primero un viaje por HTTP/1.1 para enterarse de que hay HTTP/3; con el registro
`HTTPS` del DNS, la información de por dónde ir por HTTP/3 **llega en la misma
consulta que la dirección**. Es **el mismo cambio conceptual que `CAA`**: una
entidad que representa al dominio declara una política sobre el dominio, y los
respetadores la obedecen.

### Lo que **no** cambió y sigue preguntándose

- La **jerarquía root → TLD → autoritativo**, la **delegación** por `NS` y el
  **bootstrap** con los *glue records*.
- Los **cinco campos** del Resource Record, y el **TTL en segundos**.
- Los **tipos**: `A`, `AAAA`, `NS`, `CNAME`, `MX`, `SOA`, `PTR`, `TXT`, `SRV`.
- Los **algoritmos** iterativo y recursivo, y la cadena **stub → recursivo →
  root → TLD → autoritativo**. *(Esto lo desarrolla la Clase 11.)*
- Los **puertos**: **53 UDP y 53 TCP**, y **853** para DoT.
- El **round-robin** por varios registros `A`. Sigue usándose, y Chrome y Safari
  lo aleatorizan por privacidad.

> **Consecuencia para el examen:** si te preguntan *«¿qué cambió en el DNS?»*,
> la respuesta con mejor relación esfuerzo/puntos es: **nada de la arquitectura**.
> Ni la jerarquía, ni la delegación, ni los cinco campos del RR, ni los dos
> algoritmos cambiaron desde 1987. Lo que se **agregó** es DNSSEC (autenticidad,
> subdesplegada), EDNS(0) (el mecanismo de extensión), `CAA` (política de
> certificación) y `SVCB`/`HTTPS` (política de transporte). Si en cambio te
> preguntan *«¿qué es el DNS?»*, la respuesta es **exactamente la de 2010** y está
> en el §3.1.

---

# PARTE 2 — PRÁCTICA RESUELTA

## 2.1 Ejercicio 1 — Escribir un Resource Record completo

**Enunciado.** Escribí el registro de recurso completo que le corresponde a
`www.fing.edu.uy`, que resuelve a la dirección IPv4 `132.42.120.5`, tiene clase
`IN`, y su autoritativo quiere que se cachee **media hora**.

**Resolución paso a paso.**

*Paso 1 — Identificar los cinco campos.*
```

nombre → www.fing.edu.uy. clase → IN tipo → A (porque el dato es una IPv4) TTL → media hora = 30 × 60 = 1800 segundos datos → 132.42.120.5

```
*Paso 2 — Escribirlo en la sintaxis de la zona (lo que verías en un archivo de
configuración de BIND):*
```

$ORIGIN fing.edu.uy. www 1800 IN A 132.42.120.5

```
*Paso 3 — Escribirlo en la sintaxis canónica completa (lo que va por el cable):*
```

www.fing.edu.uy. 1800 IN A 132.42.120.5 │ │ │ │ └─ rdata: 4 bytes de IPv4 │ │ │ └── type: 2 bytes, valor 1 (A) │ │ └──────── class: 2 bytes, valor 1 (IN) │ └──────────────────── TTL: 4 bytes, valor 1800 └──────────────────────────────────────────────── name: 18 bytes en cable (ver §1.5)

```
*Paso 4 — Calcular cuántos bytes ocupa en la respuesta.*
```

nombre: 18 bytes (3'www'4'fing'3'edu'2'uy'0) tipo: 2 bytes clase: 2 bytes TTL: 4 bytes rdlength: 2 bytes (cuántos bytes ocupa el rdata) rdata: 4 bytes (la IPv4) ──────────────────────────────────────── Total RR: 32 bytes

```
**Interpretación física:** un registro `A` ocupa **32 bytes**. Cuando hacés un
`dig` de un dominio con 50 registros `A`, estás moviendo 1600 bytes de datos, más
los nombres de las preguntas. **Un paquete UDP de 512 bytes (el mínimo DNS) alcanza
para unas 12 entradas `A`.** Por eso las respuestas grandes se cortan y hay que
usar TCP (Clase 11).

*Paso 5 — Y si el autoritativo quisiera que sea un nombre en vez de una IP:*
```

www.fing.edu.uy. 1800 IN A 132.42.120.5 www.fing.edu.uy. 1800 IN AAAA 2001:7fd:21::5 ^^^^^^^^^^^^^^^ ^^^^^ ^ ^^^ ^^^^^^^^^^^^^^^ nombre TTL clase tipo datos

(un mismo nombre SÍ puede tener varios registros del MISMO tipo: así es como se hace el balanceo)

```
### ⚠️ Errores típicos
- **Olvidar el punto final** en el nombre. En la forma canónica `fing.edu.uy` sin
  punto es un **nombre relativo**, que se resuelve con el `$ORIGIN` de la zona.
- **Confundir los bytes de `clase` y `tipo`.** Los dos son de 2 bytes, pero van en
  orden **tipo primero, clase después** en el formato de cable, aunque en la
  notación de los archivos de zona la convención es **clase primero**.
- **Poner el TTL en minutos.** La unidad es **segundos**, y siempre.

## 2.2 Ejercicio 2 — Distinguir `CNAME` de `MX` con ejemplos

**Enunciado.** Una empresa tiene estos cuatro nombres. Decí qué registro
corresponde a cada caso y **por qué no las otras opciones**.
```

a) El sitio www.fing.edu.uy se va a mudar a un servidor que se llama "portal-institucional" en el data center del ISP.

b) La Universidad tiene dos servidores de correo, smtp1 y smtp2. Quiere que smtp1 sea el preferido y que, si se cae, se use smtp2.

c) El dominio nuevo "servidor-central.uy" quiere saber a qué dirección IP corresponde.

d) La empresa quiere que el correo a "no-spam"@fing.edu.uy no se acepte desde ningún servidor, y que lo declare públicamente con un TXT.

```
**Resolución paso a paso.**

*a) El sitio se muda de servidor → **`CNAME`**.*
```

www.fing.edu.uy. 1800 IN CNAME portal-institucional.isp.uy.

```
*Por qué: el objetivo es que **`www.fing.edu.uy` deje de ser un nombre terminal**
para pasar a ser **un alias** de otro. El cliente resuelve `www.fing.edu.uy`,
obtiene un nombre, y vuelve a resolver. El beneficio es que, cuando el servidor
se muda otra vez, **solo cambiás un registro**, en un solo lugar.

⚠️ **Las dos trampas del `CNAME`:**
```

1. Un nombre con CNAME NO puede tener ningún otro registro. www.fing.edu.uy. CNAME x.isp.uy. www.fing.edu.uy. MX 10 mail. ← ¡PROHIBIDO! → respuesta del servidor: "CNAME y otro tipo en el mismo nombre"
2. Un CNAME NO puede existir en la raíz. ni en un TLD, ni en la zona directa de un dominio.

```
*b) Preferencia entre servidores de correo → **`MX`**.*
```

fing.edu.uy. 1800 IN MX 10 smtp1.fing.edu.uy. fing.edu.uy. 1800 IN MX 20 smtp2.fing.edu.uy.

```
*Por qué: el `MX` no devuelve una IP, devuelve **un par (preferencia, nombre de
servidor)**. El `10` y el `20` **no son puertos**: son prioridades, y **menor es
mejor**. El MTA del emisor mira **todos** los MX, ordena por preferencia, y manda
primero al más bajo.
```

10 smtp1 ◄── se intenta PRIMERO 20 smtp2 ◄── si smtp1 responde 4.x.x o 5.x.x (error temporal), se reintenta con smtp2

```
⚠️ **La trampa clásica del ejercicio:** un `MX` **no puede contener
`132.42.120.5`**. Si se pone un número donde va el nombre del servidor, la mayoría
de los MTAs lo rechaza y el correo **no llega**. Hay que poner **siempre un
nombre**, y ese nombre se resuelve aparte con un `A` o `CNAME`.
```

❌ fing.edu.uy. MX 10 132.42.120.5 (no: es un nombre, no una IP) ✅ fing.edu.uy. MX 10 smtp1.fing.edu.uy. ✅ smtp1.fing.edu.uy. A 132.42.120.5 (el nombre se resuelve aparte)

```
*c) Un nombre nuevo → dirección IP → **`A`**.*
```

servidor-central.uy. 1800 IN A 132.42.60.10

```
*Por qué: es exactamente para eso. Y el `A` **no puede tener CNAME ni MX ni nada
más**: `A` es un dato terminal, no un alias.

*d)Declaración de política de correo → **`TXT`**.*
```

fing.edu.uy. 1800 IN TXT "v=spf1 -all"

```
*Por qué: el `TXT` guarda **texto libre**, y la industria lo usa para **SPF**
(*Sender Policy Framework*), que declara qué servidores tienen permiso de enviar
correo en nombre del dominio. `-all` significa «ninguno». `v=spf1 ...` es la forma
moderna; `TXT` también se usa para **DKIM** y para **verificación de propiedad**
ante Google Workspace o Microsoft 365.

### ⚠️ Errores típicos
- **Poner una IP en un `MX`.** Es el error número uno de configuración de correo.
- **Confundir el número del `MX` con un puerto.** `MX 10` es prioridad, `MX 25`
  sería prioridad 25, no puerto 25. Y ojo: **puede haber un `MX 25`**, que no tiene
  nada que ver con SMTP.
- **Meter un `CNAME` en la raíz.** No existe, y muchos paneles de hosting lo
  ofrecen igual: es un error del panel, no tuyo.

## 2.3 Ejercicio 3 — ¿Por qué un `CNAME` no puede existir en la raíz?

**Enunciado.** Explicá por qué la especificación prohíbe un `CNAME` en la raíz del
espacio de nombres y en la zona directa de un TLD. No es un capricho: la razón
está en el diseño de la **delegación**.

**Resolución paso a paso.**

*Paso 1 — Recordar cómo funciona la resolución.*

Para resolver un nombre hay que **bajar** por la jerarquía: root → TLD → autoritativo.
En cada paso el servidor de arriba **no tiene la respuesta**, pero tiene el `NS` del
siguiente nivel.

*Paso 2 — Ver qué haría un CNAME en la raíz.*
```

Si la raíz tuviera: . IN CNAME otro.nombre.

Entonces el root, ante CUALQUIER consulta, tendría que: 1. devolver una referencia a otro.nombre 2. el cliente resuelve otro.nombre 3. para resolver otro.nombre, vuelve a preguntar a la raíz 4. la raíz vuelve a decir "andá a otro.nombre" 5. ∞ bucle infinito

```
*Paso 3 — La razón real: la raíz es el **índice**, no un **nodo**.*
```

La raíz cumple el PAPEL de un índice: le dice al cliente a quién preguntarle. Un CNAME significa "soy otro nombre". Si el índice fuera un alias, dejaría de ser un índice: no habría ningún nodo al que ir.

╔═══════════════

```
*Paso 4 — Por qué el prohibition es por **nombre** y no por **zona**.*

La regla es: **un nombre con `CNAME` no puede tener ningún otro registro, y el
nombre `.` (ni el nombre de un TLD, que es de una sola etiqueta) es el nombre del
propio servidor de la zona.** Es una regla de **coherencia**, no de seguridad: si
`uy. IN CNAME x.` fuera legal, entonces el `NS` de `uy.` tendría que convivir con
el `CNAME`, y la delegación quedaría indefinida.

**Consecuencia práctica que se pregunta en los exámenes:** como `.uy` no puede
tener `A`, **no existe un servidor web en la raíz de `.uy`**: `http://uy/` no
funciona. Por eso existen los portales de TLD como `nic.uy` o `portal.uy`.

### ⚠️ Errores típicos
- **Decir que es por seguridad.** No: es por **coherencia de la delegación**. La
  seguridad aparece cuando un atacante mete un `CNAME` en un nombre que sí puede
  tenerlo: es el **DNS hijacking**, que es un ataque por contenido, no una
  violación de la especificación.
- **Olvidar que la prohibición aplica también a los TLD.** `com. IN CNAME ...` es
  tan ilegal como `. IN CNAME ...`.

---

# PARTE 3 — PROBLEMAS DE EXAMEN DE FING

## 3.1 ⭐ Examen 2010-02, Pregunta 1a (8 puntos) — qué es el DNS y su función

> **Fuente:** Examen `eirc1002` / `sirc1002` — 2 de febrero de 2010.
> **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1002.pdf

### Enunciado (texto literal del examen)

> **Pregunta 1 (8 puntos)**
>
> a) Defina qué es el DNS y cuál es su función.

### Solución oficial (del PDF, textual)

> El protocolo DNS (Domain Name System) es un protocolo de capa de aplicación que
> permite realizar la traducción de nombres de hosts a direcciones IP. También se
> denomina DNS a la base de datos jerárquica distribuida que almacena los mapeos
> entre nombres y direcciones IP.

### Resolución, con mi razonamiento

La oficial son **dos frases**. Para los 8 puntos hay que **desarmarlas**, porque cada
palabra de las dos frases es un punto:
```

"protocolo de capa de aplicación" │ └─▶ el DNS es un protocolo como HTTP o SMTP: mensajes, peticiones y respuestas, sobre UDP (puerto 53). NO es un algoritmo, no es una base de datos relacional, no es "un servicio" en el sentido de un proceso servidor y ya.

"traducción de nombres de hosts a direcciones IP" │ └─▶ la FUNCIÓN. Con nombre: "www.fing.edu.uy". Sin nombre: 132.42.120.5.

"base de datos jerárquica distribuida" │ ├─▶ base de datos: guarda datos y los sirve a preguntas. ├─▶ jerárquica: la información se reparte en niveles │ (root → TLD → autoritativo), no está toda junta. └─▶ distribuida: los datos están repartidos en MUCHAS máquinas (los 13 root, los cientos de TLD, los 2 autoritativos de cada dominio). No hay una máquina central.

"mapeos entre nombres y direcciones IP" │ └─▶ el contenido: tablas de correspondencia. "Mapeo" = *mapping*.

**Respuesta redactada, lista para copiar en el examen:**

> **a)** El DNS (Domain Name System) es un **protocolo de capa de aplicación** cuyo
> propósito es permitir la **traducción de nombres de host a direcciones IP**: los
> usuarios y las aplicaciones trabajan con nombres (por ejemplo `www.fing.edu.uy`),
> que son legibles y jerárquicos, y el DNS devuelve la dirección IP correspondiente
> (por ejemplo `132.42.120.5`), que es la que permite enviar paquetes.
>
> Estrictamente hablando, «DNS» designa **dos cosas a la vez**. Primero, es un
> **espacio de nombres jerárquico**: los nombres se organizan en un árbol
> (`.uy` → `edu` → `fing.edu.uy` → `www.fing.edu.uy`) resuelto de derecha a
> izquierda, lo que permite **delegar la administración** de cada rama a una
> autoridad distinta sin que ninguna autoridad tenga que conocer la rama entera.
> Segundo, es una **base de datos jerárquica distribuida**: los datos (las
> correspondencias nombre–dirección, llamadas *mapeos*) están repartidos en miles de
> servidores organizados en esa misma jerarquía. Cada servidor solo guarda los datos
> de su zona y **no conoce los de las demás**, y responde a las consultas de las
> zonas inferiores.
>
> No puede ser un archivo central porque (i) la tabla completa no entra en cada host
> y habría que redistribuirla en cada alta o baja, (ii) sería un punto único de
> falla y de compromiso, y (iii) no habría forma de validar quién autorizó cada
> nombre. La **distribución** resuelve las tres cosas: escala, tolerancia a fallas y
> mantenimiento.

**Lo que agregaría yo y la oficial no dice** (y es lo que más suma, porque
demuestra que entendiste el diseño):

1. **La jerarquía es lo que hace posible la distribución.** Es un árbol en el que
   cada nodo conoce **solo a sus hijos** y delega en ellos la autoridad: es la
   propiedad de un padre que reparte entre cada hijo la porción que le corresponde.
   Un árbol plano (una sola tabla con todos los nombres) no escalaría; uno con copia
   completa en cada nodo, tampoco.
2. **El nombre es un espacio de nombres, no solo una etiqueta.** Los nombres son
   **únicos y jerárquicos**, y eso es lo que permite
   que `edu.uy` y `edu` puedan coexistir sin colisión: son ramas distintas del árbol.
3. **El DNS es bidireccional conceptualmente, pero en la práctica es unidireccional.**
   Nombre → IP (`A`) es lo que todo el mundo usa. IP → nombre (`PTR`) existe pero
   casi solo sirve para logística y para los servidores de correo. La resolución
   inversa **no se puede usar para validar** nada, porque no está garantizada.

> **Nota contra la oficial:** la oficial es precisa pero **corta**: enumera las dos
> definiciones y no explica **por qué** tiene que ser jerárquica y distribuida. Si
> el corrector es humano, agregar la justificación del archivo central da puntos;
> si es automático, la oficial es el texto que hay que reproducir.

## 3.2 Examen 2010-02, Pregunta 1b (8 puntos) — iterativa vs. recursiva

> **Fuente:** el mismo examen, apartado b).

### Enunciado (texto literal del examen)

> b) Explique la diferencia entre las consultas iterativas y recursivas al DNS.

### Solución oficial (del PDF, textual)

> En una consulta iterativa un host le hace una consulta a su DNS local. El DNS local
> en ese momento deberá encargarse de resolver la consulta reenviándola a todos los
> servidores DNS que sean necesarios. Comienza por consultar la raíz, y cada DNS que
> consulte (que no tenga la respuesta) le indicará cuál es el servidor siguiente (más
> abajo en la jerarquía) al que debe preguntarle.
>
> En una consulta recursiva el host le hace una consulta a su DNS local. El DNS local

```
> preguntará a un servidor raíz. Si la raíz no conoce la respuesta, él mismo le
> preguntará al servidor que corresponda que esté más abajo en la jerarquía. Si este
> servidor no conoce la respuesta, se la preguntará al siguiente que esté más abajo
> en la jerarquía. Luego de que obtienen la respuesta, se volverá hacia atrás en la
> recursión para devolver el resultado al que consultó originalmente.

### Resolución, con mi razonamiento

**Un solo ejemplo mental y las dos definiciones salen solas.** Alguien pregunta
«¿dónde está la Farmacia?»:
```

═══ ITERATIVO ═══ (el que pregunta va y viene)

```
YO ──"¿sabés dónde está la Farmacia?"──▶ VECINO
YO ◀──"no sé, preguntale a la Farmacia de Pellegrini 1230"── VECINO
YO ──"¿sabés...?"──▶ FARMACIA DE PELLEGRINI 1230
YO ◀──"sí, es acá"──
(yo hice 2 viajes)
```

═══ RECURSIVO ═══ (el que pregunta se hace cargo de todo)

```
YO ──"¿sabés dónde está la Farmacia?"──▶ VECINO
VECINO ──"¿sabés...?"──▶ FARMACIA DE PELLEGRINI 1230
FARMACIA ──"sí, es acá"──▶ VECINO
VECINO ──"es acá"──▶ YO
(yo hice 1 viaje, pero el vecino hizo 3)
```

**La diferencia en una frase:**

| | Iterativa | Recursiva |
|---|---|---|
| **Quién sigue preguntando** | **el que preguntó** | **el que recibió la pregunta** |
| **Qué devuelve el que no sabe** | «no sé, **preguntale vos** a este» | **pregunta él** y vuelve
| **Quién carga con el trabajo** | el cliente (4 viajes de ida y vuelta) | el servidor (1 viaje pa
| **Quién la usa** | el **resolver recursivo** (el del ISP), entre servidores | el **host**, contra

> **El error típico de examen, y es el que la propia oficial pisea:** decir que «el
> host le hace una consulta iterativa a su DNS local». No. El **stub resolver del
> host** hace una consulta **recursiva** a su servidor de nombres local: «resolveste
> esto por mí y traeme la respuesta final». Lo que el **servidor local** hace hacia
> afuera son consultas **iterativas**: le pregunta al root, el root le dice
> «preguntale al `.uy`», y así. **Esto no cambió desde 1987.**

**Respuesta redactada, lista para copiar en el examen:**

> **b)** Se distinguen por **quién se hace cargo de seguir consultando** cuando el
> servidor consultado **no** conoce la respuesta.
>
> En una **consulta iterativa**, el servidor que no tiene la respuesta **no hace
> nada por su cuenta**: devuelve al cliente una **referencia** («ese nombre está
> delegado a este otro servidor, andá a preguntarle») y el cliente, que es el que
> empezó, **sigue bajando por la jerarquía** hasta llegar a un servidor que sí tenga
> la respuesta. El cliente hace, entonces, **cuatro viajes**: uno a cada nivel.
>
> En una **consulta recursiva**, el servidor que no tiene la respuesta **se hace cargo
> él mismo** de consultar al nivel inferior, y así sucesivamente hasta obtener la
> respuesta final (o un error). Cuando la tiene, **vuelve hacia atrás por la
> recursión** devolviéndola a quien le consultó. El cliente hizo **un solo viaje**.
>
> En la práctica, el **cliente** (el *stub resolver* del sistema operativo) le
> hace una consulta **recursiva** a su **servidor de nombres local** (el del ISP),
> que a su vez hace consultas **iterativas** al root, al TLD y al autoritativo.
> La iterativa existe para que **un solo servidor recursivo** pueda hacer el trabajo
> pesado y cachear todo, en vez de hacer que cada cliente de Internet repita los
> mismos cuatro viajes.

**Sobre el matiz de la oficial que está bien dicho y hay que repetir:** la oficial
dice, para la recursiva, que «**luego de que obtienen la respuesta, se volverá hacia
atrás en la recursión**». Esa frase es la clave: **recursión** significa que el
**call stack** crece a medida que se baja y se **desarma** a medida que se sube. No es un concepto

## 3.3 Examen 2007-07, Pregunta 4 (8 puntos) — función del DNS y protocolo involucrado

> **Fuente:** Examen `eirc0707` / `sirc0707` — julio de 2007.

Este problema se desarrolla **entero en la Clase 11**, junto con la caché y el TTL
(que es lo que aporta de nuevo). Acá se deja el planteo, que es de §1.1 y §1.2:

- **Función del DNS:** traducir nombres a direcciones IP, resolviendo de derecha a
  izquierda sobre un espacio de nombres jerárquico y distribuido.
- **Protocolo involucrado:** el DNS es un protocolo de **capa de aplicación** que
  viaja sobre **UDP puerto 53** en el caso normal, y sobre **TCP puerto 53** cuando
  la respuesta no entra en un datagrama o cuando se pide una **transferencia de
  zona** (`AXFR`).
- **Mensaje:** 12 bytes de cabecera (ID, flags, QDCOUNT, ANCOUNT, NSCOUNT,
  ARCOUNT) más cuatro secciones (Question, Answer, Authority, Additional).

---

# PARTE 4 — CIERRE

```
## 📌 Resumen en 10 líneas

1. El DNS existe porque los humanos **no memoramos IPs** y porque una IP no dice
   quién es el dueño, ni si el servidor está vivo, ni quién lo administra.
2. **No puede ser un archivo central** por tres razones independientes: la tabla no
   entra en cada host, sería un punto único de falla **y** de compromiso, y no hay
   forma de validar quién autorizó cada nombre.
3. El DNS es **dos cosas**: un **espacio de nombres jerárquico** (cómo se nombran y
   quién tiene autoridad) y una **base de datos distribuida** (qué corresponde a
   qué). La primera dice a quién preguntarle, la segunda qué preguntar.
4. El nombre se resuelve **de derecha a izquierda** porque la autoridad va de la
   raíz hacia abajo, y solo la raíz sabe qué TLDs existen.
5. El **root** (13 identidades `a`–`m`, IPv4 por *anycast*) **no conoce los
   dominios**: solo sabe quién es el TLD de cada uno. El **TLD** tampoco conoce los
   dominios: sabe quién es su autoritativo.
6. La **delegación** se hace con registros `NS`, y los **glue records** (registros
   `A` con el nombre de un servidor de nombres) existen para **romper el círculo**:
   sin ellos, el nombre del servidor autoritativo no se podría resolver.
7. Un **Resource Record** tiene **cinco campos**: nombre, clase (`IN`), tipo, TTL
   (4 bytes) y datos. El TTL son **segundos**, y lo define el **autoritativo** de
   ese registro.
8. `A` y `AAAA` son IP; `NS` son servidores de nombres; `CNAME` es un alias; `MX`
   es el servidor de **correo** con un número de **preferencia**; `SOA` es el
   control de zona (serial, refresh, retry, expire, minimum); `PTR` es la inversa;
   `TXT` texto libre; `SRV` servicio con puerto; `CAA` qué CA puede emitir.
9. El **stub resolver** del SO consulta **recursivamente** al **resolver local**,
   que a su vez consulta **iterativamente** a root, TLD y autoritativo.
10. **Esto no cambió desde 1987**: ni la jerarquía, ni los dos algoritmos, ni los
    puertos (53 UDP y TCP). Lo que cambió es la privacidad (DoH, Clase 11).

## ✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

**1.** ¿Por qué el DNS no puede ser un archivo central? Dá las **tres** razones.

<details><summary>Respuesta</summary>

1. **Escala:** la tabla completa tiene una entrada por host (hoy del orden de miles de millones). A
2. **Punto único de falla:** si el archivo o el servidor que lo sirve se cae, **todo Internet deja
3. **Mantenimiento:** no hay forma de validar **quién autorizó** cada nombre a registrarse, porque

La solución del DNS es la **distribución jerárquica**: particionar los datos por zona, replicar ca

</details>

**2.** ¿Qué diferencia hay entre un **nombre** y un **dominio**? ¿Y entre un
**dominio** y un **host**?

<details><summary>Respuesta</summary>

En la práctica se usan como sinónimos, pero técnicamente:

- **Nombre** (*name*): cualquier etiqueta del espacio de nombres. `www` es un nombre, `fing.edu.uy
- **Dominio**: un nombre **delegado**, es decir del que alguien es responsable. `fing.edu.uy` es u
- **Host** (*host*): un nombre que además tiene un registro `A` o `AAAA` y por lo tanto **designa

Por eso el enunciado del `2010-02 P1a` habla de «nombres de hosts»: es el nivel más bajo y más con

</details>

**3.** ¿Qué sabe el root server y qué **no** sabe? ¿Cuántos hay?
```

```
<details><summary>Respuesta</summary>

Hay **13 identidades lógicas**, nombradas de `a.root-servers.net` a `m.root-servers.net`, y cada u

**Sabe:** los registros `NS` (y `A`) de todos los **TLD**, unos 1500 entradas. Sabe quién administ

**NO sabe:** los dominios. Jamás supo qué es `fing.edu.uy`, ni cuántos hay, ni qué IP tienen. Pregu

El número **13** es un artefacto histórico de 1987 (elegido por el DARPA), congelado después por l

</details>

**4.** ¿Qué es un **glue record** y por qué existe? ¿Por qué a veces hay que
duplicar información?

<details><summary>Respuesta</summary>

Un **glue record** es un registro `A` (o `AAAA`) con el **nombre de un servidor de nombres**, guar

Existe para **romper el círculo del bootstrap**: si el `.uy` te dice «el autoritativo de `fing.edu

La información está **duplicada** (el mismo `A` existe en el glue del `.uy` y en la zona `fing.edu

El costo es que se puede **desincronizar**: si el autoritativo cambia de IP y nadie actualiza el g

</details>

**5.** Escribí un RR completo para `mail.fing.edu.uy` → `132.42.120.9`, clase `IN`,
cacheable 1 hora. Decí además cuántos bytes ocupa.

<details><summary>Respuesta</summary>
```

nombre clase tipo TTL datos mail.fing.edu.uy. IN A 3600 132.42.120.9 ```

La TTL es `1 hora = 60 × 60 = 3600 segundos`. En la zona se escribiría `$ORIGIN fing.edu.uy.` y `mail 3600 IN A 132.42.120.9`.

Tamaño: nombre `4mail4fing3edu2uy0` = 1+4 + 1+4 + 1+3 + 1+2 + 1 = **18 bytes**; más tipo 2 + clase 2 + TTL 4 + rdlength 2 + rdata 4 = **32 bytes** en total.

**6.** Un dominio tiene `MX 10 smtp1.uy` y `MX 20 smtp2.uy`. Si `smtp1` está caído y `smtp2` **no acepta** correo de esa persona, ¿qué pasa? ¿Dónde está el problema?

▶ **Respuesta**

**7.** ¿Por qué un `CNAME` no puede existir en la raíz del espacio de nombres?

▶ **Respuesta**

**8.** ¿Por qué no se puede resolver un nombre «de derecha a izquierda» si el nombre se lee de izquierda a derecha?

▶ **Respuesta**

## 🔗 Referencias

- **Examen `eirc1002` / `sirc1002`** — Pregunta 1 completa (8 pts): qué es el DNS y su función (a) y diferencia entre consultas iterativas y recursivas (b)
- **Examen `eirc1012` / `sirc1012`** — Pregunta 1a (8 pts): el TTL del RR, quién lo define, a qué refiere y en qué unidades
- **Examen `eirc1007` / `sirc1007`** — Pregunta 3 (8 pts): resolución iterativa completa sin caché (desarrollada en la Clase 11)
- **Examen `eirc0707` / `sirc0707`** — Pregunta 4 (8 pts): función del DNS y protocolo involucrado
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.4 («El sistema de nombres de dominio»), §7.4.2 (el espacio de nombres jerárquico)
- **RFC 1034** — *Domain Names: Concepts and Facilities* (1987): el diseño completo, que **no cambió**
- **RFC 1035** — *Domain Names: Implementation and Specification* (1987): el formato del mensaje y de los RR
- **RFC 6891** — EDNS(0) (2013, *Internet Standard*): añade el pseudo-registro `OPT`; base de DNSSEC, CAA y SVCB/HTTPS
- **RFC 8659** — CAA, *Certification Authority Authorization* (2019; reemplaza a RFC 6844)
- **RFC 4033-4035** — DNSSEC (2005) · **RFC 5155** — raíz firmada (2010) · **RFC 8080** — EdDSA · **RFC 8901** — *multi-signer*
- **RFC 9460** — SVCB/HTTPS (2023): el DNS transporta política de transporte
- **IANA**, *Root Zone Database* — la lista real de los 13 root servers y los TLDs existentes
- **Clase siguiente:** 11 — DNS: iterativo vs. recursivo, caché y TTL